

REPORT

Knowledge Distillation in Computer Vision – Teacher-Student method

Vedran Mitić 2123

Introduction

The idea of knowledge distillation has traveled a path — not so much in time as in research — from a concept seen as a model compression method to its “power” of regularization. Although pioneering work exists on this topic, the name of this method was first formalized in 2015 in [1], and to this day it retains its basic concept — the existence of two models, a teacher and a student.

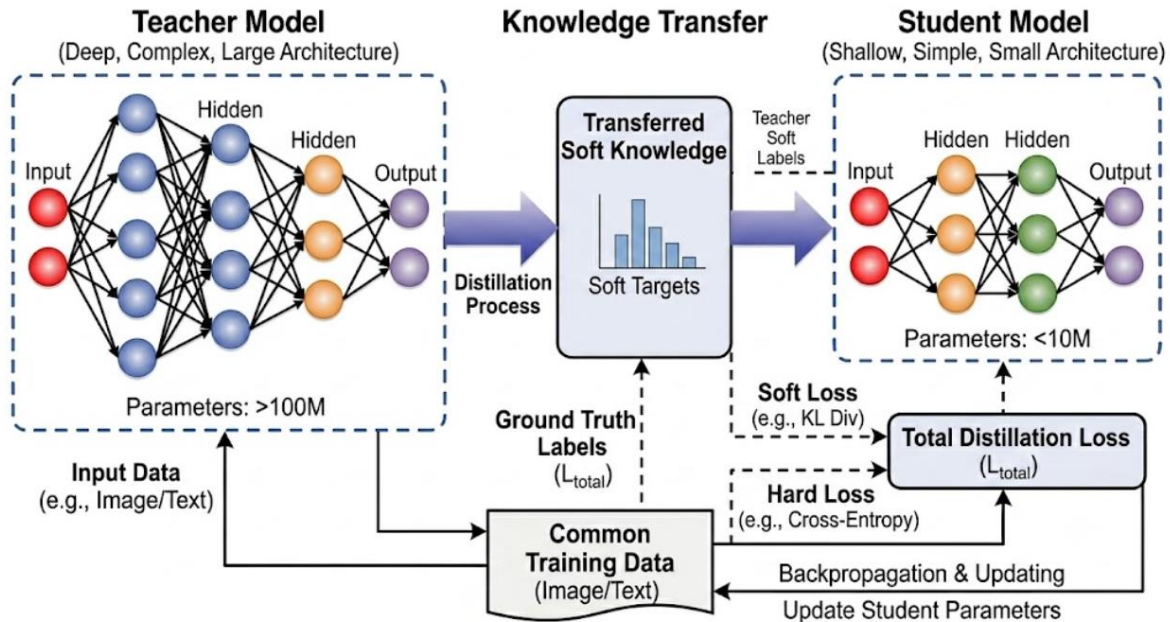


Figure 1. Generalized overview of the KD method

The teacher is a large model, usually one of the state-of-the-art models trained on large datasets, while the student is a smaller model that aims to imitate the teacher's knowledge with similar results. The teacher, often referred to in the literature as the cumbersome model, is first trained in order to extract data structures and representations (details and similarities between classes). The learned knowledge is then distilled into the student, which in this way very often gains an even broader picture of what it is learning compared to what the teacher had while learning. An overview of the method is shown in Figure 1.

The method itself consists of three key components:

1. Knowledge (according to its source):

- Response-based knowledge – knowledge derived from the last (output) layer of the model;
- Feature-based knowledge – knowledge extracted from the intermediate (hidden) layers;
- Relation-based knowledge – knowledge based on the mutual relationships and structures of the data itself.

2. Distillation algorithm (according to the learning method, Figure 2):

- Offline distillation – separate teacher training → knowledge extraction → student training;
- Online distillation – simultaneous training of teacher and student;

- Self-distillation – the same model plays the role of both teacher and student.

3. Teacher-student architectures:

Although there is no established classification of teacher-student architectures in the literature, it can intuitively be concluded that a more complex state-of-the-art model is chosen for the teacher, and a model with reduced architectural capacity for the student [2]. This taxonomy can be further expanded, and with the transition to the transformer architecture, supplemented with significant innovations such as DeiT [3].

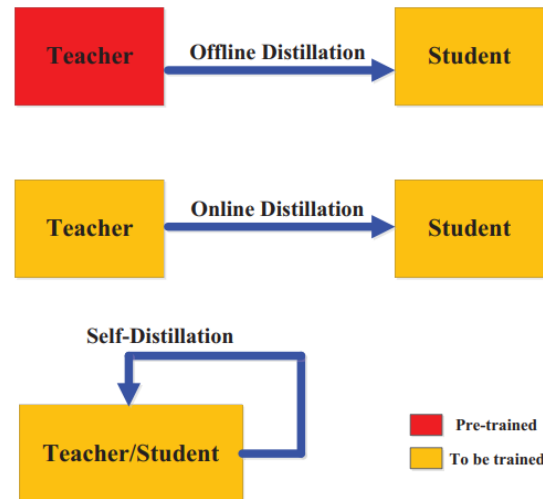


Figure 2. "Different types of distillation. The red color 'Pre-trained' indicates that the networks were trained before distillation, while the yellow color 'To be trained' indicates that the networks learn during distillation." [2]

The practical part of the paper covers offline response-based distillation.

Response-based KD – G. Hinton

The term "knowledge distillation" first appears in [1], as the name for the process of transferring knowledge from an already trained, cumbersome model to a small model. The main challenge that existed in this field up to that point was how to actually change the form of the model while retaining the same knowledge. Additionally, the goal is also to surpass the results that would be achieved by a small model trained using the same principle and dataset as the large model. The solution presented in the form of KD is, in fact, the simple transfer of "soft targets" to the student, on the basis of which it would perform additional training. Soft targets are nothing more than a vector of predicted class probabilities at the teacher's output, which serve as target labels for the small model. In practice, this would mean that the student does not only have information about the "hard targets," i.e., the already known class-membership labels (0 or 1), but also information about how the teacher perceives the data itself (e.g., an image). Thus, if the teacher saw an image of a dog with a 30% probability that it is a wolf and 5% that it is a cat, the student would learn not only that the image shows a dog, but also that a dog shares certain similarities with a wolf. For the same example, if the teacher saw the image with a 0.5% probability that it is a car, the student would also learn that a dog is much more similar to a cat than to a car. In this way, the distinction between classes becomes more understandable and richer than merely forcing optimization toward hard labels. These probabilities are adjusted by changing the value of the temperature T in the softmax layer, such that as T increases, their distribution becomes more uniform. [1] also points out that

as the entropy of the probabilities increases, they provide more information per sample and much less variance in the gradient, meaning that training requires a smaller dataset than the original one and a higher learning rate, which considerably speeds up training.

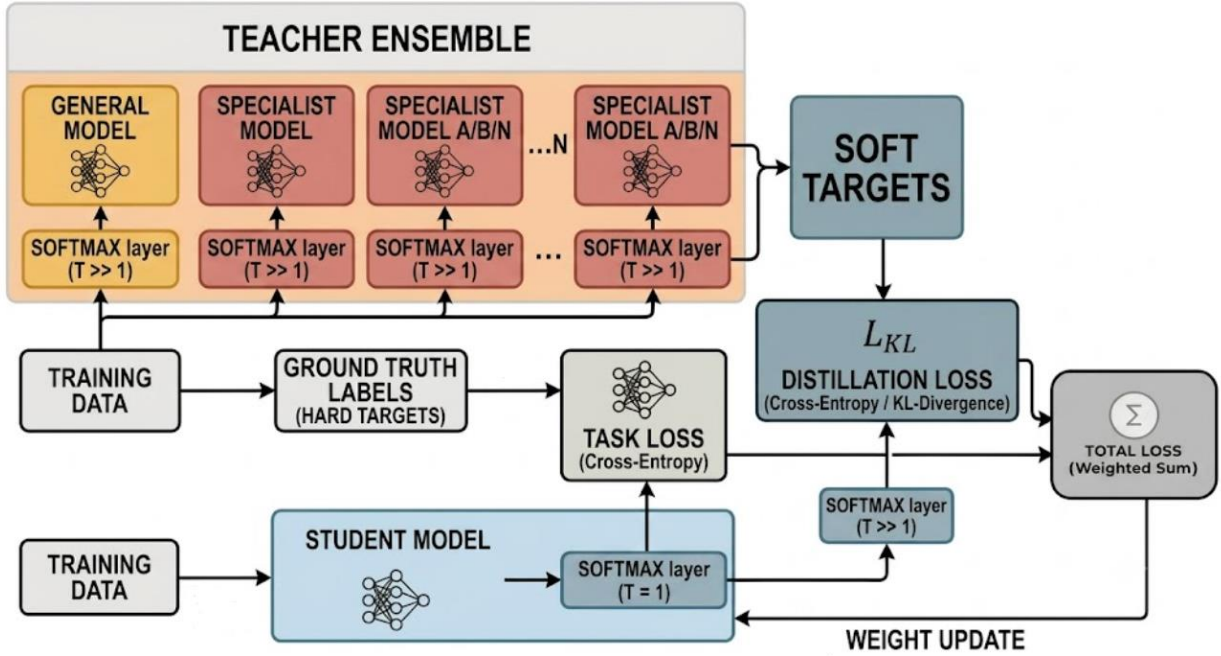


Figure 3. Graphical representation of the KD architecture

When the number of classes is very large, [1] proposes that the cumbersome teacher be composed of an ensemble of models consisting of one large, powerful model trained on all the data — the “Generalist” — and several “Specialist” models. Training is first carried out on the Generalist, whose errors are observed and used to form clusters of classes that were problematic for it to classify. Specialists (also large models initialized with the Generalist’s weights) are then created, each trained only on the problematic classes, so that each Specialist is responsible for one cluster, thereby learning the “fine” differences that the Generalist missed. This creates fine-grained classes, i.e., subclass types of certain classes, or subclasses that differ from one another by some hidden details — for example, car models within the car class. To reduce overfitting in the Specialists, half of the training data consists of samples from the special subclasses, while the other half is randomly sampled from the rest of the original dataset. To reduce bias, the logit of the “dustbin” class (a class representing the combination of all classes that the Specialist does not deal with) can be increased by the logarithm of the proportion by which the Specialist’s class was “oversampled” (how many times more often the special classes appear in the specialist’s training set compared to their actual frequency in the original dataset) [1].

The soft target passed on to the student is represented by the vector of joint probability distributions of the Generalist and the active Specialists — q — obtained by minimizing the sum of their KL divergences:

$$KL(p^g, q) + \sum_{m \in A_k} KL(p^m, q) \quad (1)$$

where p^g represents the Generalist's probability distribution, and p^m the probability distribution of all the special classes from m , plus the dustbin class.

Finally, the student's learning is achieved by minimizing the final loss function, which represents a weighted sum of the knowledge from the true labels and the knowledge from the teacher (“hidden” knowledge, “Dark Knowledge”):

$$L_{total} = (1 - \alpha)L_{CE}(y, \sigma(z_s)) + \alpha T^2 L_{KL}(q, \sigma\left(\frac{z_s}{T}\right)) \quad (2)$$

where α is a hyperparameter that determines the weight given to each of the loss functions, L_{CE} is the classification loss function (cross-entropy) between the true labels y and the output of the softmax function applied to the student's logits $\sigma(z_s)$, and L_{KL} is the distillation loss function (Kullback-Leibler divergence) between the teacher q (the minimizer of (1)) and the softmax function applied to the student's logits $\sigma\left(\frac{z_s}{T}\right)$ at $T > 1$. The distillation loss is multiplied by the factor T^2 because, as the temperature increases, the gradient magnitude decreases by $\frac{1}{T^2}$. A depiction of the method is shown graphically in Figure 3.

The main problem with Hinton's KD was the “vanishing” gradient in deeper student architectures relative to the teacher. However, this was soon overcome by new methods and network architectures such as feature-based KD or “skip” (ResNet) connections.

PRACTICAL PART

The practical part was carried out using Google Colab.

Dataset

The dataset used is EPT100, consisting of 100 classes of microscopic insect images, divided into training (8538 images), validation (3368 images), and test (5152 images) sets. The validation set is missing images for 4 classes (140, 159, 179, and 181), so special attention must be paid to setting up the code correctly so that samples from these classes are properly mapped during training. All images are 512×512 in size.

Since Colab is used, it is best to first connect to Google Drive, on which a .ZIP file of the dataset (Dataset.zip) has previously been placed, and then use code to transfer that file from the drive to Colab's local filesystem and unpack it there directly. This is the fastest approach, since otherwise reading images from the drive later would be too slow. After unpacking, and before running the following cells, the previously listed empty folders must be manually deleted from the val_512 folder.

Model selection

EfficientNet-B2 was chosen as the teacher model so that the results could be compared with existing results for this dataset.

Three different models were chosen for the students:

- EfficientNet-B0 – a smaller version of the teacher model. This model is expected to achieve the best results compared to the others due to its compatibility with the teacher.
- TinyCustom CNN – a hand-built model consisting of a total of 9 convolutions, modeled after MobileNet. The architecture is shown in Figure 4.
- MobileNet V3 – the best-known model for edge devices.

The reason edge-device-friendly models were used is that, besides attempting to demonstrate the effectiveness of the KD technique, this is also an attempt to demonstrate its direct applicability in the field of compressing knowledge onto smaller devices. Different model architectures were chosen in order to observe the effectiveness of the technique when the student has the same architecture as the teacher model, and when it has a different one.

Hyperparameters

The teacher is set to always receive images at the original 512×512 resolution — both during training and during KD. Students receive images at a resolution of 224×224, primarily due to their smaller capacity, and also to simulate edge devices whose cameras would likely be of lower quality. Images are resized to 224×224 using bilinear interpolation. The batch size is 16.

The learning rate is 10^{-4} , except for TinyCustom CNN, where it is set to 10^{-3} since the model was built by hand. Weight decay is 10^{-4} .

The number of epochs is initially set to 30, since that is roughly the average time for which Colab credits will not be exhausted. This parameter is naturally increased each time a checkpoint is reached.

The knowledge distillation parameters are $\alpha = 0.7$ and $T = 3$ ($T = 10$).

Settings

The teacher model uses pretrained weights on ImageNet-1K and is fine-tuned by unfreezing the classifier and the last layer of EfficientNet. This was done to save training time for the teacher, which is the largest model. All student models were trained from scratch.

Knowledge distillation is configured so that the alpha hyperparameter is assigned to the KD loss, meaning that this loss accounts for 70% of the total loss. The KD loss function is KL divergence, while cross-entropy is used with the hard labels.

The model evaluation metrics are Top-1 and Top-5 accuracy, although Top-5 can largely be disregarded.

Early Stopping is set to 7 for the teacher and 15 for the students. The check is performed with respect to the loss value rather than accuracy, due to the possibility of overfitting. The loss is considered improved if the validation loss decreases by more than 0.001 (the min_delta hyperparameter).

The optimizer is AdamW, and the learning rate scheduler is CosineAnnealing.

The following augmentations were applied to the training set: Random Horizontal Flip, Random Vertical Flip, Color Jitter, Random Rotation. The data was normalized according to the ImageNet standard.

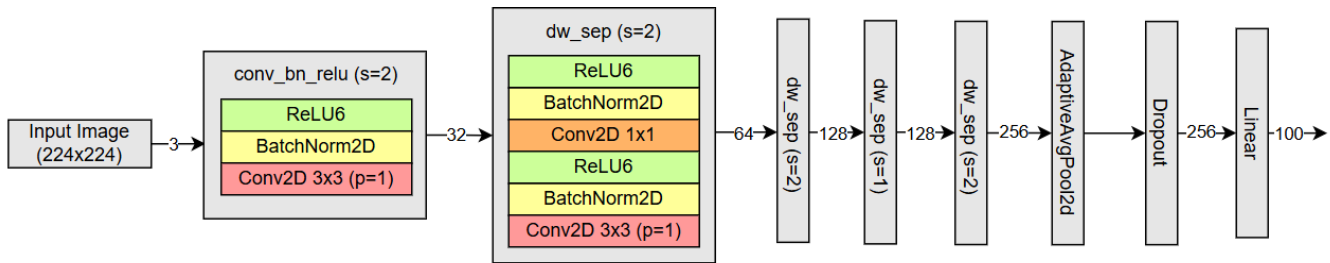


Figure 4. TinyCustom CNN architecture

Results

Model	T = 3 and $\alpha = 0.7$							T = 10 and $\alpha = 0.7$							Params (M)	Size (MB)
	Training & Validation					Test		Training & Validation					Test			
	Top-1	Top-5	Best Val Loss	Epochs	Early Stopping	Top-1	Top-5	Top-1	Top-5	Best Val Loss	Epochs	Early Stopping	Top-1	Top-5		
Teacher (EfficientNet-B2)	98.58%	100%	0.044	30	5	98.43%	99.92%	—	—	—	—	—	—	—	7.91	30.3
Student (EfficientNet-B0) HL	92.06%	99.56%	0.2769	50	15	91.01%	99.53%	—	—	—	—	—	—	—	4.18	16.1
Student (EfficientNet-B0) KD	94.11%	99.20%	0.6186	113	15	93.21%	98.97%	94.37%	99.23%	0.4129	119	15	93.32%	98.89%	4.18	16.1
Student (TinyCustomNet) HL	90.11%	99.23%	0.3619	51	15	89.09%	99.34%	—	—	—	—	—	—	—	0.09	0.4
Student (TinyCustomNet) KD	90.58%	98.93%	0.8804	74	15	88.94%	98.76%	93.22%	99.32%	0.6202	112	15	91.38%	99.36%	0.09	0.4
MobileNetV3 HL	89.31%	99.44%	0.3571	106	15	87.62%	99.30%	—	—	—	—	—	—	—	1.63	6.3
MobileNetV3 KD	89.51%	98.90%	0.7645	105	15	88.35%	98.89%	89.48%	98.82%	0.5929	120	15	88.16%	98.72%	1.63	6.3

Table 1. Training, validation, and test results of the models
Italic and Bold – best result among all models
Bold – best result within the student model subgroup

Training

The knowledge distillation process did not manage to push the performance of the smaller models to the high 98% accuracy range, but it certainly succeeded in one respect — surpassing the performance of models trained solely on hard labels. The most visible difference is with EfficientNet-B0, at +2.05% accuracy for $T = 3$. This was expected given the similarity of its architecture to that of the teacher. TinyCustom Net and MobileNet V3 increased their accuracy by +0.47% and +0.20% respectively. An even greater improvement can be seen when the temperature is increased to $T = 10$: +2.31% for EfficientNet-B0, +3.11% for TinyCustom Net, and +0.17% for MobileNet V3. Increasing the temperature also allowed for longer student training, as reflected in the increased number of epochs. Finally, increasing the temperature significantly reduced the validation loss for all models. The effect of temperature on the predictions can be seen in Figure 7.

Models trained solely on hard labels not only have the worst accuracy results, but also considerably less room (fewer epochs) for training (learning). The exception is MobileNet V3, which proved to be the “most resistant” to knowledge distillation. Also, models trained in this way have lower validation loss, which is a consequence of the loss function itself. In models using KD, the total loss also includes the KL divergence, which is multiplied by T^2 (the square of the temperature). Because of this, the KD loss is always numerically higher than the plain cross-entropy used by models trained on hard labels.

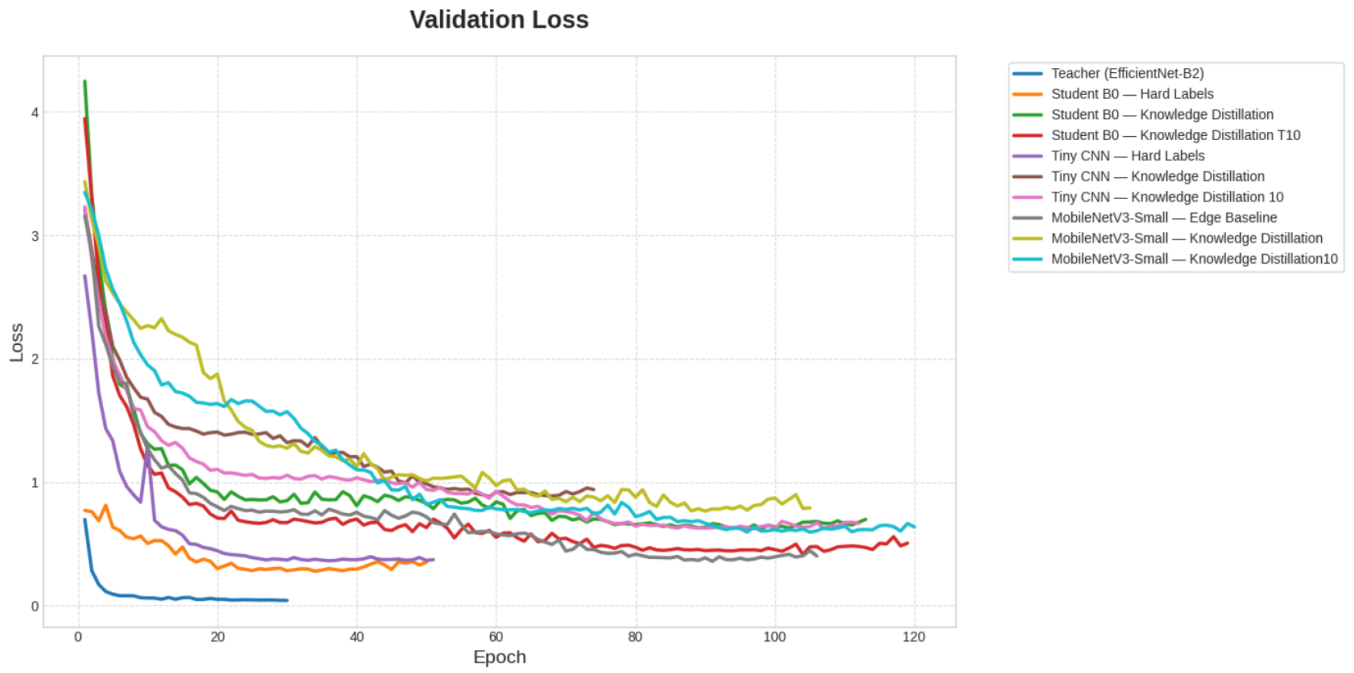


Figure 5. Chart of validation loss change across epochs

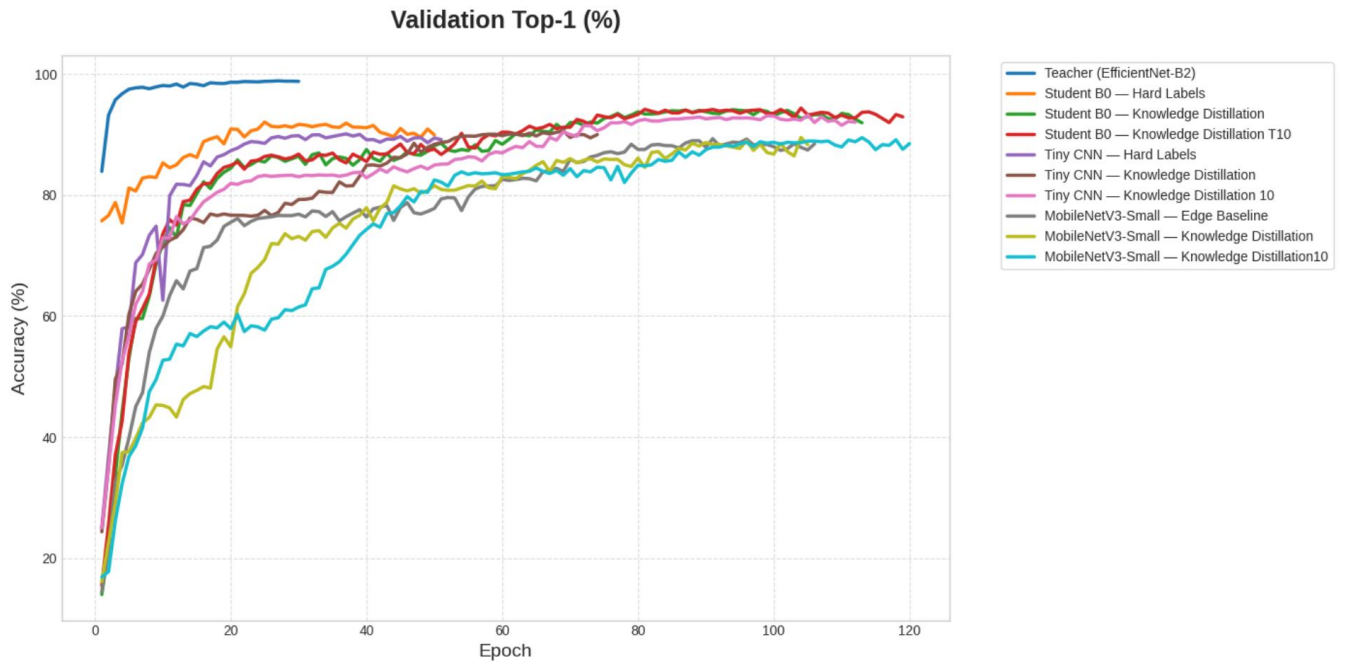


Figure 6. Chart of validation accuracy change across epochs

Inference

At temperature $T = 3$, knowledge distillation contributed to improvements for EfficientNet-B0 (+2.20%) and MobileNet V3 (+0.73%), while, interestingly, accuracy for TinyCustom Net decreased by -0.16% . With the temperature increased to $T = 10$, the accuracy for TinyCustom Net this time increased by +2.44% compared to distillation at $T = 3$, and by +2.29% compared to training on hard labels; for MobileNet V3 the accuracy remained higher than its baseline but lower than at $T = 3$, while EfficientNet-B0 managed to

surpass its previous results with 93.32% accuracy. The number of misclassifications on the test set is given in Table 2.

Model	Errors	Total
Teacher (EfficientNet-B2)	81	5152
Student (EfficientNet-B0) HL	463	5152
Student (EfficientNet-B0) KD T = 3	350	5152
Student (EfficientNet-B0) KD T = 10	344	5152
Student (TinyCustomNet) HL	562	5152
Student (TinyCustomNet) KD T = 3	570	5152
Student (TinyCustomNet) KD T = 10	444	5152
MobileNetV3 HL	638	5152
MobileNetV3 KD T = 3	600	5152
MobileNetV3 KD T = 10	610	5152

Table 2. Number of misclassifications on the test set

It should be taken into account that among these errors, there are 25 images that no model managed to classify correctly, representing 0.49% of the test data. With $T = 10$, this number of images is 22. These images likely represent objective noise in the form of labeling errors or extremely poor-quality photos. Table 3 shows that the number of errors shared with the teacher remains stable, ranging from 43 to 52 images for all distilled models, indicating that the students consistently inherit the hard-to-solve samples from the teacher's set of misclassifications.

Model	Teacher EfficientNet- B2	EfficientNet- B0 HL	EfficientNet- B0 KD	EfficientNet- B0 KD T=10	TinyCustomNet HL	TinyCustomNet KD	TinyCustomNet KD T=10	MobileNetV3 HL	MobileNetV3 KD	MobileNetV3 KD T=10
Teacher EfficientNet-B2	81									
EfficientNet-B0 HL	39	463								
EfficientNet-B0 KD	46	243	350							
EfficientNet-B0 KD T=10	43	227	250	344						
TinyCustomNet HL	53	259	216	218	562					
TinyCustomNet KD	50	223	205	204	347	570				
TinyCustomNet KD T=10	51	204	195	189	304	377	444			
MobileNetV3 HL	51	310	256	245	327	304	251	638		
MobileNetV3 KD	52	290	241	234	323	314	274	375	600	
MobileNetV3 KD T=10	50	286	253	239	310	296	254	384	357	

Table 3. Shared errors across all models (Number of images that both models misclassified)

Having singled out a few misclassified samples (from the previously mentioned set of 22 images) and correctly classified samples (from the set of 3791 images that all models got right), and by observing the assigned probability for the predicted incorrect class in Table 4 and the assigned probability for the predicted correct class in Table 5, it can be noted that the distilled models generally predict with lower confidence than those trained solely on hard labels, which is evidence of this method's regularizing power. While the model trained on hard labels shows extreme and rigid confidence in misclassification (often over 90%), the KD models show considerably greater restraint. This is an expected phenomenon, since the KL divergence teaches the student to mimic the teacher's softened probabilities (Figure 7), actively penalizing it whenever it extremely favors a single class. As the temperature increases to $T = 10$, due to

better generalization and the overall increase in model accuracy, the confidence with which the students predict the correct class also increases (Table 5), while at the same time retaining protection against excessive overconfidence in errors compared to models trained solely on hard labels.

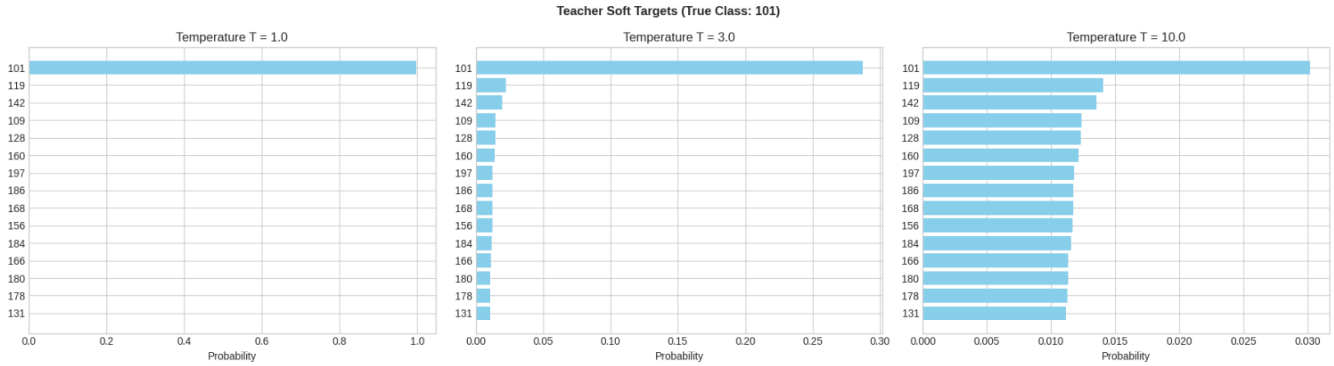


Figure 7. Effect of temperature change on the teacher's outputs

Model	True class = 119		True class = 123		True class = 123	
	Predicted class	Assigned probability	Predicted class	Assigned probability	Predicted class	Assigned probability
Teacher (EfficientNet-B2)	137	68.7%	154	97.3%	124	87.3%
(EfficientNet-B0) HL	137	99.8%	154	92.2%	124	84.2%
(EfficientNet-B0) KD T = 3	137	59.4%	154	78.5%	124	67.0%
(EfficientNet-B0) KD T = 10	137	83.5%	154	79.1%	124	58.8%
(TinyCustomNet) HL	115	57.0%	154	95.4%	124	38.2%
(TinyCustomNet) KD T = 3	137	42.5%	154	42.7%	124	21.5%
(TinyCustomNet) KD T = 10	137	30.2%	154	68.3%	124	27.9%
MobileNetV3 HL	137	99.8%	154	79.7%	124	89.8%
MobileNetV3 KD T = 3	137	70.4%	154	36.9%	124	17.2%
MobileNetV3 KD T = 10	137	80.3%	124	31.2%	124	81.0%

Table 4. Examples of misclassified samples

Model	True class = 108	True class = 119	True class = 107	True class = 110	True class = 138
Teacher (EfficientNet-B2)	100.00%	100.00%	99.97%	99.83%	99.79%
(EfficientNet-B0) HL	97.96%	99.08%	68.74%	97.50%	99.99%
(EfficientNet-B0) KD T = 3	47.21%	64.36%	39.46%	67.95%	71.13%
(EfficientNet-B0) KD T = 10	75.00%	86.53%	85.37%	82.29%	84.23%
(TinyCustomNet) HL	91.13%	98.51%	93.60%	94.38%	97.11%
(TinyCustomNet) KD T = 3	52.69%	73.48%	64.01%	66.62%	51.88%
(TinyCustomNet) KD T = 10	86.84%	83.10%	92.96%	74.54%	66.73%
MobileNetV3 HL	98.77%	99.99%	99.30%	99.89%	98.86%
MobileNetV3 KD T = 3	37.46%	77.37%	82.75%	66.09%	71.07%
MobileNetV3 KD T = 10	69.99%	90.66%	46.70%	81.16%	91.68%

Table 5. Examples of correctly classified samples

Using Grad-CAM visualization to gain insight into how the models perceive the image before classification produces the result shown in Figure 8. It is clear that the teacher extracts the insect from the image almost perfectly, given its complexity and the resolution at which it views the image. On the other hand, the base models trained without distillation (EfficientNet-B0 and MobileNetV3) focus

predominantly on the head and front part of the insect, showing a scattered focus that also captures irrelevant background above and below the object. As the temperature is increased to $T = 10$, the attention of the distilled models becomes considerably more restrained toward the background and begins to correctly follow the contours of the insect itself. This trend is especially noticeable in the TinyCustom Net model, which, through distillation, managed to successfully redirect and distribute its attention along the insect's entire body, closely approximating the attention map possessed by the teacher.

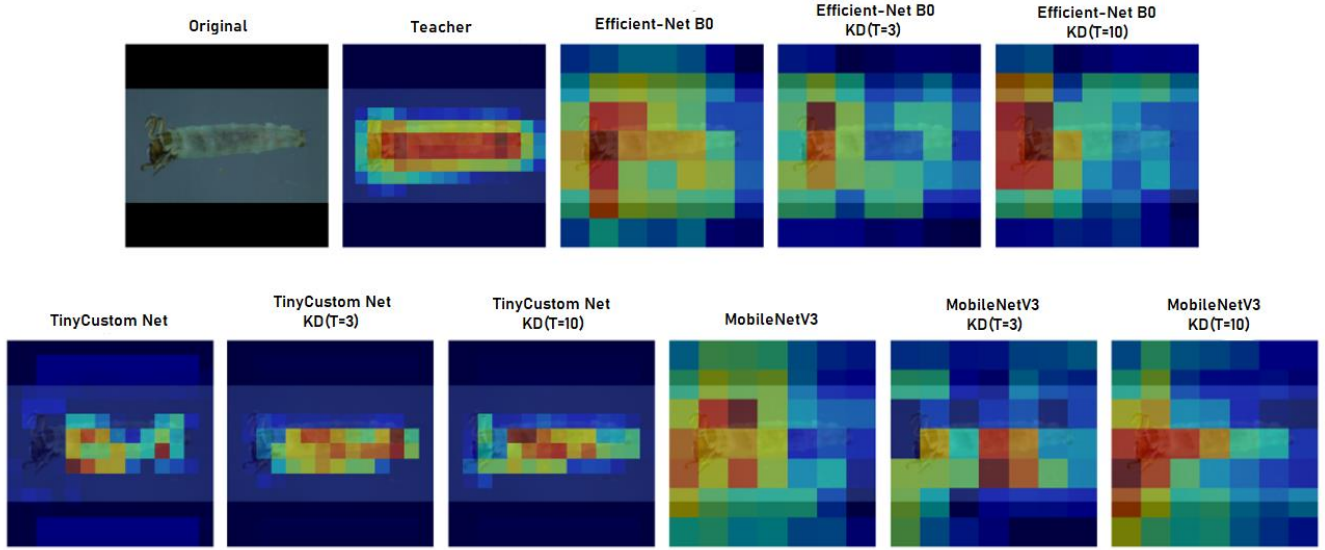


Figure 8. Grad-CAM visualization

One of the most important characteristics for edge models is inference speed, given in Table 6. Logically, speed is inversely proportional to model size, i.e., the number of parameters. Among the currently trained models, the most optimal choice would be TinyCustom Net Knowledge Distillation with $T = 10$. This model managed to exceed 90% accuracy while being 8.16 times faster (8.31 times on CPU) than the same EfficientNet-B0, and 12.22 times faster (93.68 times on CPU) than the teacher.

Model	Top-1	GPU latency (ms)	CPU latency (ms)	Params (M)
Teacher (EfficientNet-B2)	98.427795%	12.381394	374.145792	7.841894
(EfficientNet-B0) HL	91.013199%	8.308280	36.116261	4.135648
(EfficientNet-B0) KD $T = 3$	93.206522%	11.038508	37.871001	4.135648
(EfficientNet-B0) KD $T = 10$	93.322981%	8.266258	33.190138	4.135648
(TinyCustomNet) HL	89.091615%	1.016233	3.878694	0.091044
(TinyCustomNet) KD $T = 3$	88.936335%	1.134076	3.823178	0.091044
(TinyCustomNet) KD $T = 10$	91.381988%	1.013339	3.993804	0.091044
MobileNetV3 HL	87.616460%	5.299416	12.963986	1.620356
MobileNetV3 KD $T = 3$	88.354037%	7.082109	14.010763	1.620356
MobileNetV3 KD $T = 10$	88.159938%	7.149558	9.435546	1.620356

Table 6. Model inference time

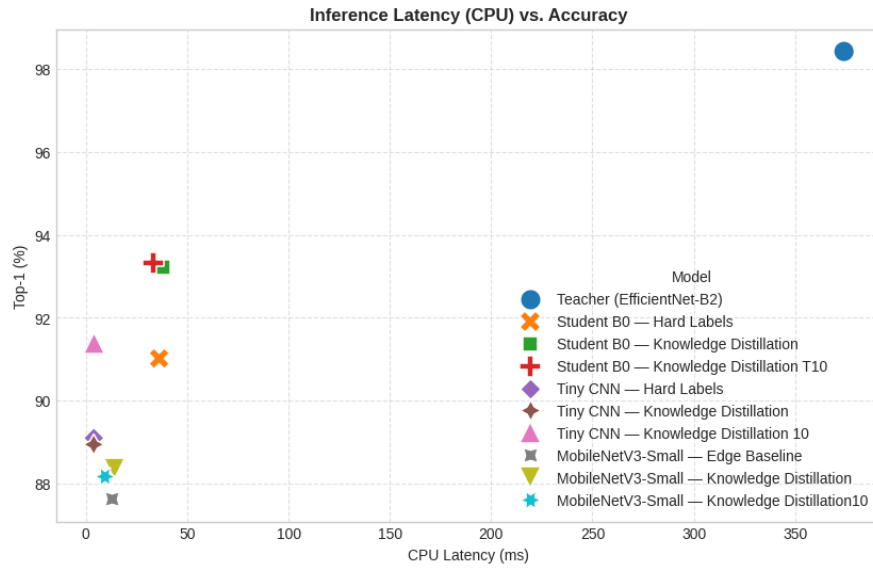


Figure 9. Chart of the relationship between inference time and model accuracy

Conclusion

For the given dataset, the response-based knowledge distillation method confirmed the theory by achieving an accuracy improvement for all student models and making them more generalized compared to the versions trained solely on hard labels. To more closely reach the performance of the teacher, EfficientNet-B2, future work should attempt to use pretrained weights for the student models as well, in the same way this was done for the teacher. Regarding TinyCustom Net, further experimentation with hyperparameters could be conducted first, in order to obtain a better base model before repeating the distillation process. Additionally, finer tuning of the temperature and alpha parameters could help find the most optimal solution for the given dataset. Furthermore, it would be more effective to create a larger capacity gap between the teacher and student models, i.e., to choose either a much larger model than EfficientNet-B2 or a much smaller one than EfficientNet-B0 (here, the teacher is only 1.9 times larger). In this work, this was slightly compensated for by the fact that the teacher views images at twice the resolution.

Finally, it should be noted that today, in practice, due to its shortcomings, response-based knowledge distillation alone is rarely used; instead, it is combined with feature-based distillation or other complementary methods, which contributes to improving the accuracy and robustness of the models.

References

- [1] Hinton, G., Vinyals, O. & Dean, J. (2015). Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531.
- [2] Gou, J., Yu, B., Maybank, S. J. & Tao, D. (2020). Knowledge distillation: A survey. arXiv preprint arXiv:2006.05525.
- [3] Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A. & Jégou, H. (2020). Training data-efficient image transformers & distillation through attention. arXiv preprint arXiv:2012.12877.
- [4] Romero, A., Ballas, N., Kahou, S. E., Chassang, A., Gatta, C. & Bengio, Y. (2014). FitNets: Hints for thin deep nets. arXiv preprint arXiv:1412.6550.
- [5] Park, W., Kim, D., Lu, Y. & Cho, M. (2019). Relational knowledge distillation. arXiv preprint arXiv:1904.05068.
- [6] Xu, G., Liu, Z., Li, X. & Loy, C. C. (2020). Knowledge distillation meets self-supervision. arXiv preprint arXiv:2006.07114.
- [7] Cheng, Y., Wang, D., Zhou, P. & Zhang, T. (2018). Model compression and acceleration for deep neural networks: The principles, progress, and challenges. *IEEE Signal Proc Mag* 35(1): 126–136.
- [8] Wu, J., Leng, C., Wang, Y., Hu, Q. & Cheng, J. (2016). Quantized convolutional neural networks for mobile devices. In: *CVPR*.
- [9] Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). Mobilenets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861.